

# Research Project: Weak binary semaphore

Team members: Tianbin Wang, Kehao Zhang

Class: CE356 in 26Spring

## 1. Background and Definition

A significant challenge arises when dealing with weak binary semaphores. Unlike "strong" semaphores, which typically use a FIFO queue to manage blocked processes, weak semaphores provide no guarantee regarding the order in which processes are unblocked. This lack of deterministic scheduling introduces the risk of starvation—a scenario where a process is perpetually denied access to the critical section, even if the resource becomes available.

## 2. Project Plan

We plan to first construct a mutual exclusion algorithm that relies entirely on weak binary semaphores as the synchronization primitive. We will then model the protocol in TLA+, capturing process states, semaphore states, and the specific transitions required to enter and leave the critical section.

## 3. Ideas and Approaches

A key aspect of our approach is modeling the number of processes,  $n$ , as a configurable parameter of the system. We will utilize TLC to verify essential Safety properties, primarily ensuring strict mutual exclusion (i.e., no two processes occupy the critical section simultaneously).

Once safety is established, we introduce Weak Fairness constraints. Since weak binary semaphores do not guarantee a FIFO (First-In-First-Out) order for the blocked queue, we use TLC to examine Liveness properties—specifically, the assertion that "every process requesting entry to the critical section will eventually succeed."

When TLC detects a violation of a liveness property, it generates a counter-example. We will conduct a detailed analysis of this "execution trace" to reveal how, under weak semaphore conditions, a process may suffer from starvation due to the "infinite overtaking" by other processes. This allows us to not only verify the algorithm's correctness but also visually identify the extreme concurrency scenarios that lead to system failure.

Based on the feedback provided by TLC, we will iterate on the algorithm (e.g., by introducing additional gating logic or priority mechanisms). Finally, we will clearly define the Progress Guarantees of the model within safety-critical architectures—explicitly distinguishing which properties are mathematically guaranteed and which risks must be mitigated through higher-level system scheduling.